

AHDS Code Library #7 — Caméra USB

Modernisation d'une application FASM après plus de 20 ans

Version héritage 2004/2005 → Version Windows 10 modernisée en 2026

Ce projet présente l'évolution d'une petite application de capture d'image par caméra USB, initialement développée en assembleur FASM au début des années 2000, puis modernisée en 2026 afin de fonctionner sous Windows 10 avec une caméra USB/UVC.

1. Version héritage — 2004 / 2005

La première version a été développée en FASM (Flat Assembler) avec l'API Win32 et l'ancienne pile de capture vidéo Video for Windows / AVICAP32. Elle permettait de connecter une caméra, d'afficher un aperçu vidéo en direct et de capturer une image.

Cette architecture correspondait aux générations de Windows utilisées à cette époque. Le projet a notamment fonctionné sous Windows XP. Avec l'évolution des systèmes Windows et des caméras USB, cette méthode de capture est devenue inadaptée aux environnements plus récents.

2. Modernisation — 2026

En 2026, environ 22 ans après la création de la version originale, le projet a été repris avec un objectif précis : conserver l'application principale en assembleur FASM, tout en remplaçant la partie de capture devenue obsolète par une architecture compatible avec Windows 10.

FASM / Win32 API → DLL C++ → Microsoft Media Foundation → Caméra USB/UVC

3. Architecture hybride FASM + C++

L'interface utilisateur et la logique principale restent développées en FASM Win32. Le moteur moderne de gestion de la caméra est placé dans une bibliothèque dynamique camlib1.dll développée en C++ avec Microsoft Visual Studio 2019 et Microsoft Media Foundation.

La DLL expose les fonctions nécessaires au programme FASM, notamment le démarrage de la capture, l'arrêt de la caméra et la prise de photo. L'application récupère dynamiquement les adresses de ces fonctions à l'aide de LoadLibrary() et GetProcAddress().

4. DLL intégrée dans l'exécutable

Une particularité importante du projet est que camlib1.dll n'a pas besoin d'être distribuée séparément avec l'application. La DLL compilée est intégrée directement dans les ressources du fichier EXE FASM.

Au démarrage, l'exécutable extrait automatiquement la DLL dans le répertoire temporaire de Windows (%TEMP%), la charge en mémoire, puis utilise ses fonctions pour communiquer avec la caméra. À la fermeture de l'application, la bibliothèque temporaire peut être déchargée et supprimée.

Le résultat final est donc une application facilement transportable sous la forme d'un seul fichier EXE, tout en conservant un moteur de capture moderne développé en C++.

5. Fonctionnement de la version Windows 10

- Connexion à une caméra USB/UVC.
- Initialisation de la capture avec Microsoft Media Foundation.
- Affichage de l'aperçu vidéo directement dans la fenêtre de l'application FASM.
- Conversion du flux vidéo vers un format RGB32 exploitable pour l'affichage GDI.
- Démarrage et arrêt de la caméra depuis l'interface.
- Capture d'une image au format BMP.

6. Évolution de l'architecture



Figure 1 — Évolution de l'application : architecture héritage 2004/2005 et modernisation Windows 10 en 2026.

Le schéma met en évidence le changement de la couche de capture vidéo sans abandonner l'application FASM d'origine. La version héritage utilisait directement AVICAP32 / Video for Windows, tandis que la version modernisée confie l'acquisition vidéo à un moteur C++ basé sur Media Foundation. Cette séparation permet de conserver l'interface assembleur tout en utilisant une technologie de capture adaptée à Windows 10.

7. Contenu prévu dans le dépôt GitHub

Le dépôt GitHub permet de conserver les deux générations du projet et de comparer directement leur architecture. Il peut contenir :

- le code source FASM de la version héritage destinée aux anciennes générations de Windows, notamment Windows XP ;
- l'exécutable de la version héritage ;
- le code source FASM de la version modernisée pour Windows 10 ;
- l'exécutable autonome de la version Windows 10 ;
- camlib1.dll ;
- le projet et le code source C++ de la DLL, développés avec Microsoft Visual Studio 2019 ;
- le fichier .DEF utilisé pour conserver des noms d'exportation simples et stables ;
- la documentation et les illustrations présentant l'évolution du projet.

8. Compilation et outils

La partie assembleur est développée avec FASM (Flat Assembler). Pour reconstruire le projet, l'utilisateur peut installer FASM séparément depuis sa source officielle. La partie caméra moderne est développée en C++ avec Microsoft Visual Studio 2019 et Microsoft Media Foundation.

Le dépôt fournit ainsi les sources nécessaires pour étudier la version historique, comprendre l'intégration EXE/DLL et reconstruire la version modernisée.

9. Technologies utilisées

FASM · x86 Assembly · Win32 API · C++ · Visual Studio 2019 · DLL · **Microsoft Media Foundation** · UVC · **USB Camera** · **GDI** · **Windows XP** · **Windows 10**

10. GitHub

Code source, versions héritage et Windows 10, DLL et projet Visual Studio :

<https://github.com/Anfiska2023>

Conclusion

Ce projet illustre la modernisation progressive d'un logiciel legacy sans supprimer son identité technique d'origine. La logique et l'interface FASM sont conservées, tandis qu'une couche C++ / Media Foundation assure la compatibilité avec une caméra USB/UVC sous Windows 10. La comparaison des deux versions permet également de montrer concrètement l'évolution des API Windows et des méthodes d'intégration logicielle sur plus de deux décennies.

Code FASM pour Windows 10

format PE GUI 4.0

entry codestart

include 'C:\fasmw17164\INCLUDE\win32a.inc'

; =====

; CONSTANTES ET IDENTIFIANTS DES CONTRÔLES

; =====

IDD_MAIN = 100

ID_RUN = 201

ID_OFF = 202

ID_PHOTO = 203

IDC_CANVAS = 204

IDR_CAM_DLL = 500 ; Identifiant de la ressource DLL embarquée

section '.data' data readable writeable

hInstance dd ?

hDlgMain dd ?

hCamDll dd ?

hCanvas dd ?

pfnStartCapture dd ?

pfnStopCapture dd ?

pfnTakePhoto dd ?

_fnStart db 'StartCameraCapture', 0

_fnStop db 'StopCameraCapture', 0

_fnPhoto db 'TakeCameraPhoto', 0

szTempPath rb MAX_PATH

szDllPath rb MAX_PATH

_dllFileName db 'camlib1.dll', 0

_errExtractMsg db 'Impossible de prendre le fichier camlib1.dll dans les ressources !', 0

_errDllMsg db 'Impossible de charger camlib1.dll !', 0

_errCaption db 'Erreur', 0

section '.code' code readable executable

codestart:

 invoke GetModuleHandle, 0

 mov [hInstance], eax

; 1. Extraction de camlib1.dll depuis les ressources vers le dossier temporaire %TEMP%

 call ExtractDllFromResources

 test eax, eax

 jz .err_extract

; 2. Chargement de la DLL extraite

 invoke LoadLibrary, szDllPath

 mov [hCamDll], eax

 test eax, eax

 jz .err_dll

; 3. Importation des adresses des fonctions

 invoke GetProcAddress, [hCamDll], _fnStart

```
mov    [pfnStartCapture], eax
```

```
invoke GetProcAddress, [hCamDll], _fnStop
```

```
mov    [pfnStopCapture], eax
```

```
invoke GetProcAddress, [hCamDll], _fnPhoto
```

```
mov    [pfnTakePhoto], eax
```

```
; 4. Lancement de la boîte de dialogue principale
```

```
invoke DialogBoxParam, [hInstance], IDD_MAIN, HWND_DESKTOP, MainDlg, 0
```

```
; 5. Déchargement de la DLL et suppression du fichier temporaire à la fermeture
```

```
cmp    [hCamDll], 0
```

```
jz     .clean_file
```

```
invoke FreeLibrary, [hCamDll]
```

```
.clean_file:
```

```
invoke DeleteFile, szDllPath
```

```
invoke ExitProcess, 0
```

```
.err_extract:
```

```
invoke MessageBox, 0, _errExtractMsg, _errCaption, MB_ICONERROR
```

```
invoke ExitProcess, 1
```

```
.err_dll:
```

```
invoke MessageBox, 0, _errDllMsg, _errCaption, MB_ICONERROR
```

```
invoke ExitProcess, 1
```

```
; =====
```

; PROCÉDURE D'EXTRACTION DE LA DLL DEPUIS LES RESSOURCES

; =====

proc ExtractDllFromResources

push ebx esi edi

local hRes dd ?, hResData dd ?, dwSize dd ?, hFile dd ?, dwWritten dd ?

; Obtention du chemin du dossier temporaire %TEMP%

invoke GetTempPath, MAX_PATH, szTempPath

test eax, eax

jz .fail

; Construction du chemin complet : %TEMP%\camlib1.dll

invoke lstrcpy, szDllPath, szTempPath

invoke strcat, szDllPath, _dllFileName

; Recherche de la ressource dans l'exécutable

invoke FindResource, [hInstance], IDR_CAM_DLL, RT_RCDATA

mov [hRes], eax

test eax, eax

jz .fail

; Chargement de la ressource en mémoire

invoke LoadResource, [hInstance], [hRes]

mov [hResData], eax

test eax, eax

jz .fail

; Obtention de la taille de la DLL en octets

invoke SizeofResource, [hInstance], [hRes]

```
mov    [dwSize], eax
```

```
test   eax, eax
```

```
jz     .fail
```

```
; Obtention du pointeur vers les données en mémoire
```

```
invoke LockResource, [hResData]
```

```
mov     esi, eax
```

```
test   eax, eax
```

```
jz     .fail
```

```
; Création du fichier camlib1.dll sur le disque
```

```
invoke CreateFile, szDllPath, GENERIC_WRITE, 0, 0, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, 0
```

```
mov     [hFile], eax
```

```
cmp     eax, INVALID_HANDLE_VALUE
```

```
je     .fail
```

```
; Écriture des octets depuis la mémoire vers le fichier
```

```
lea     eax, [dwWritten]
```

```
invoke WriteFile, [hFile], esi, [dwSize], eax, 0
```

```
invoke CloseHandle, [hFile]
```

```
mov     eax, 1
```

```
jmp     .finish
```

```
.fail:
```

```
xor     eax, eax
```

```
.finish:
```

```
pop     edi esi ebx
```


ret

endp

; =====

; GESTIONNAIRE DE MESSAGES DE LA BOÎTE DE DIALOGUE

; =====

proc MainDlg, hdlg, msg, wparam, lparam

push ebx esi edi

cmp [msg], WM_INITDIALOG

je .wminitdlg

cmp [msg], WM_COMMAND

je .wmcommand

cmp [msg], WM_CLOSE

je .wmclose

xor eax, eax

jmp .finish

.wminitdlg:

mov eax, [hdlg]

mov [hDlgMain], eax

invoke GetDlgItem, [hdlg], IDC_CANVAS

mov [hCanvas], eax

jmp .finish

.wmcommand:

mov eax, [wparam]

and eax, 0FFFFh

```
cmp    eax, ID_RUN
je     .startbutton
cmp    eax, ID_OFF
je     .stopbutton
cmp    eax, ID_PHOTO
je     .photobutton
jmp    .finish
```

.startbutton:

```
cmp    [pfnStartCapture], 0
jz     .finish
push   [hCanvas]
call   [pfnStartCapture]
jmp    .finish
```

.stopbutton:

```
cmp    [pfnStopCapture], 0
jz     .finish
call   [pfnStopCapture]
jmp    .finish
```

.photobutton:

```
cmp    [pfnTakePhoto], 0
jz     .finish
call   [pfnTakePhoto]
jmp    .finish
```

.wmclose:

```

    cmp    [pfnStopCapture], 0

    jz     .end_dlg

    call   [pfnStopCapture]

.end_dlg:

    invoke EndDialog, [hdlg], 0


.finish:

    pop    edi esi ebx

    ret

endp

```

```

; =====

```

```

; TABLE D'IMPORTATION DES BIBLIOTHÈQUES SYSTÈME

```

```

; =====

```

```

section '.idata' import data readable writeable

```

```

library kernel, 'KERNEL32.DLL',\
    user, 'USER32.DLL'

```

```

import kernel,\
    GetModuleHandle, 'GetModuleHandleA',\
    LoadLibrary, 'LoadLibraryA',\
    GetProcAddress, 'GetProcAddress',\
    FreeLibrary, 'FreeLibrary',\
    GetTempPath, 'GetTempPathA',\
    lstrcpy, 'lstrcpyA',\
    lstrcat, 'lstrcatA',\
    FindResource, 'FindResourceA',\
    LoadResource, 'LoadResource',\

```

```
SizeofResource, 'SizeofResource',\
LockResource, 'LockResource',\
CreateFile, 'CreateFileA',\
WriteFile, 'WriteFile',\
CloseHandle, 'CloseHandle',\
DeleteFile, 'DeleteFileA',\
ExitProcess, 'ExitProcess'
```

```
import user,\
DialogBoxParam, 'DialogBoxParamA',\
EndDialog, 'EndDialog',\
GetDlgItem, 'GetDlgItem',\
MessageBox, 'MessageBoxA'
```

```
; =====
; INCLUSION DE LA DLL DANS LES RESSOURCES (Syntaxe directe)
; =====
```

```
section '.rsrc' resource data readable
```

```
directory RT_DIALOG, dialogs,\
RT_RCDATA, rcdata
```

```
resource dialogs,\
IDD_MAIN, LANG_ENGLISH + SUBLANG_DEFAULT, main_dialog
```

```
resource rcdata,\
IDR_CAM_DLL, LANG_NEUTRAL, dll_entry
```

```
dll_entry:
dd RVA dll_bytes, dll_bytes_end - dll_bytes, 0, 0
```

dll_bytes:

file 'camlib1.dll'

dll_bytes_end:

dialog main_dialog, 'Project <Image Sensor Camera Win10>', 0, 0, 350, 200, WS_CAPTION + WS_POPUP + WS_SYSMENU + DS_MODALFRAME + DS_CENTER

dialogitem 'STATIC', '', IDC_CANVAS, 10, 10, 200, 145, WS_VISIBLE + SS_SUNKEN

dialogitem 'STATIC', '&Media Foundation Engine', -1, 220, 10, 120, 8, WS_VISIBLE

dialogitem 'STATIC', '&Compatible with UVC', -1, 220, 22, 120, 8, WS_VISIBLE

dialogitem 'BUTTON', 'RUN IMAGE CAMERA', ID_RUN, 7, 165, 85, 15, WS_VISIBLE + WS_TABSTOP

dialogitem 'BUTTON', 'OFF LINE DEVICE', ID_OFF, 100, 165, 85, 15, WS_VISIBLE + WS_TABSTOP

dialogitem 'BUTTON', 'TAKE PHOTO BY CLICK', ID_PHOTO, 20, 182, 153, 13, WS_VISIBLE + WS_TABSTOP

enddialog

Code FASM pour windows XP,2000

format PE GUI 4.0

entry codestart

include '%fasminc%\win32a.inc'

IDD_MAIN = 100

WM_CAP_DRIVER_CONNECT = WM_USER + 10

WM_CAP_DRIVER_DISCONNECT = WM_USER + 11

WM_CAP_FILE_SAVEDIB = WM_USER + 25

WM_CAP_SET_PREVIEW = WM_USER + 50

WM_CAP_SET_PREVIEWRATE = WM_USER + 52

WM_CAP_SET_SCALE = WM_USER + 53

```
ID_RUN          = 201
ID_OFF          = 202
ID_PHOTO       = 203
_camtitle      db 'image camera'
```

```
_filename db 'FOTO.JPG' ; Filename
```

```
nDevice dd 0 ; Device Number -> It can range from 0 through 9
```

```
nFPS dd 1 ; Frames per second. Must be 1000/FPS. E.g. 20 FPS = 50
```

section '.data' data readable writeable

```
hInstance dd ?
```

```
hWebcam dd ?
```

section '.code' code readable executable

codestart:

```
invoke GetModuleHandle, 0
```

```
mov [hInstance], eax
```

```
invoke DialogBoxParam, eax, IDD_MAIN, HWND_DESKTOP, MainDlg, 0
```

```
invoke ExitProcess, 0
```

proc MainDlg, hdlg, msg, wparam, lparam

```
push ebx esi edi
```

```
cmp [msg], WM_INITDIALOG
```

```
je .wminitdlg
```

```
cmp [msg], WM_COMMAND
```

```
je .wmcommand
```

```
cmp [msg], WM_CLOSE
```

```
je .wmclose
```

```
xor eax, eax
```

```

jmp     .finish

.wminitdlg:

invoke  capCreateCaptureWindow, _camtitle, WS_VISIBLE + WS_CHILD, 10, 10,\
        296, 252, [hdlg], 0

mov     [hWebcam], eax

jmp     .finish

.wmcommand:

cmp     [wparam], BN_CLICKED shl 16 + ID_RUN

je      .startbutton

cmp     [wparam], BN_CLICKED shl 16 + ID_OFF

je      .stopbutton

cmp     [wparam], BN_CLICKED shl 16 + ID_PHOTO

je      .clickbutton

.wmclose:

invoke  SendMessage, [hWebcam], WM_CAP_DRIVER_DISCONNECT, _camtitle, 0

invoke  EndDialog, [hdlg], 0

.finish:

pop     edi esi ebx

return

.startbutton:

invoke  SendMessage, [hWebcam], WM_CAP_DRIVER_CONNECT, [nDevice], 0

invoke  SendMessage, [hWebcam], WM_CAP_SET_SCALE, TRUE, 0

invoke  SendMessage, [hWebcam], WM_CAP_SET_PREVIEWRATE, [nFPS], 0

invoke  SendMessage, [hWebcam], WM_CAP_SET_PREVIEW, TRUE, 0

jmp     .finish

.stopbutton:

invoke  SendMessage, [hWebcam], WM_CAP_DRIVER_DISCONNECT, _camtitle, 0

jmp     .finish

.clickbutton:

```

```
    invoke  SendMessage, [hWebcam], WM_CAP_FILE_SAVEDIB, 0, _filename
    jmp     .finish
endp
```

```
section '.idata' import data readable writeable
```

```
library kernel, 'KERNEL32.DLL',\
    user, 'USER32.DLL',\
    avicap, 'AVICAP32.DLL'
```

```
import kernel,\
    GetModuleHandle,'GetModuleHandleA',\
    ExitProcess, 'ExitProcess'
```

```
import user,\
    DialogBoxParam, 'DialogBoxParamA',\
    EndDialog, 'EndDialog',\
    SendMessage, 'SendMessageA'
```

```
import avicap,\
    capCreateCaptureWindow, 'capCreateCaptureWindowA'
```

```
section '.rsrc' resource data readable
```

```
directory  RT_DIALOG, dialogs
resource   dialogs,\
    IDD_MAIN, LANG_ENGLISH + SUBLANG_DEFAULT, main_dialog

dialog    main_dialog, 'Project <Image Sensor Camera>', 0, 0, 350, 200, WS_CAPTION + WS_POPUP +
WS_SYSMENU +\
    DS_MODALFRAME+ DS_CENTER

    dialogitem 'STATIC','&Project Building in Beer-Sheva College, ',-1,210,10,170,8,WS_VISIBLE
```


dialogitem 'STATIC','&as project for practical engineer. ',-1,220,20,170,8,WS_VISIBLE

dialogitem 'STATIC','&For start - connect camera in port USB, ',-1,210,30,170,8,WS_VISIBLE

dialogitem 'STATIC','&control button from menu of the program ',-1,210,40,170,8,WS_VISIBLE

dialogitem 'BUTTON', 'RUN IMAGE CAMERA', ID_RUN, 7, 165, 85, 15, WS_VISIBLE +
WS_TABSTOP

dialogitem 'BUTTON', 'OFF LINE DEVICE', ID_OFF, 100, 165, 85, 15, WS_VISIBLE + WS_TABSTOP

dialogitem 'BUTTON', 'TAKE PHOTO BY CLICK', ID_PHOTO, 20, 182, 153, 13, WS_VISIBLE +
WS_TABSTOP

enddialog